

Learning with AI

Topic 1: Multi-Sensor Integration & Calibration

Mausam Shrestha

CSC 494 — IoT | Spring 2026

Northern Kentucky University

Hardware Domain — How I learned to communicate with 5 different sensors simultaneously

What Problem Did I Need to Solve?

My Smart Home Health Monitor uses **5 sensors** at once:

Sensor	Protocol	Purpose
MH-Z19C	UART	CO2 (Non-Dispersive Infrared)
DC01	UART	PM2.5 particulate
MS1100	Analog	VOC (Volatile Organic Compounds)
AHT10	I2C	Temperature + Humidity
LCD1602	I2C	Display

Before this project: I had never used I2C, UART, or run multiple sensors simultaneously.

What I Didn't Know Before

Going into this project, I had significant gaps:

- Never used I2C or UART protocols
- Did not understand how multiple devices share a bus
- Had no experience reading sensor datasheets
- Never debugged hardware communication issues
- Did not understand sensor drift or calibration
- Did not know that different ESP32 boards have different pin layouts

AI helped me close every single one of these gaps.

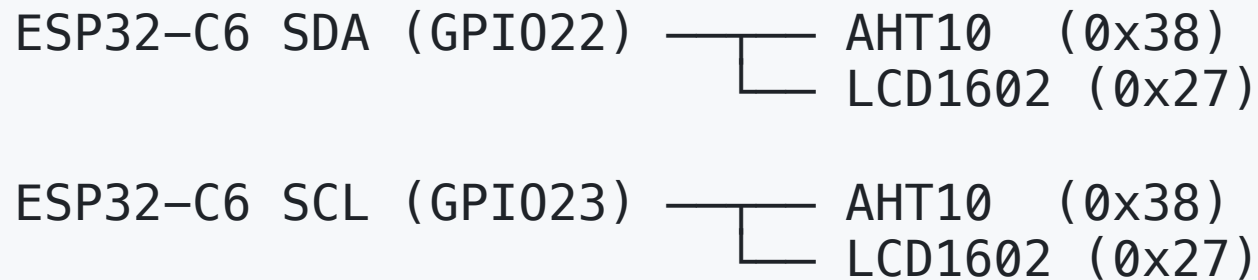
Communication Protocol 1: I2C

What I Learned

I2C (Inter-Integrated Circuit) is a **shared two-wire bus**:

- **SDA** (Serial Data)
- **SCL** (Serial Clock)

Multiple devices connect to the **same two wires** — each identified by a unique address.



AI insight: If two devices have the same address, they conflict and neither works. I

The XIAO ESP32-C6 Surprise

What AI Got Wrong — And What I Learned From It

When I couldn't get my I2C sensors to work, I asked Claude for help.

AI initially said: Use GPIO21 (SDA) and GPIO22 (SCL) — standard ESP32 pins.

Reality: The XIAO ESP32-C6 uses:

- **SDA = GPIO22** (maps to pin D4)
- **SCL = GPIO23** (maps to pin D5)

How I found the truth: Watched YouTube videos specific to XIAO ESP32-C6, then cross-referenced with the Seeed Studio schematic.

This taught me a critical lesson: **AI is trained on common hardware. Uncommon boards need verification against official datasheets.**

Communication Protocol 2: UART

What I Learned

UART (Universal Asynchronous Receiver-Transmitter) is a **point-to-point** serial protocol.

Unlike I2C, UART is **not a shared bus** — each UART sensor needs its own TX/RX pins.

For the MH-Z19C CO2 sensor:

- **Baud rate:** 9600
- **Command:** Send 9 bytes → Wait → Parse 9-byte response
- **Power:** Needs 5V (not 3.3V!) from VBUS pin

```
ESP32-C6 TX (D6)  —→  MH-Z19C RX
ESP32-C6 RX (D7)  ←—  MH-Z19C TX
VBUS (5V)         —→  MH-Z19C VIN
```

Why NDIR CO2 Matters

AI Taught Me the Difference Between Real and Estimated CO2

Cheap sensors (CCS811, MQ-135): Measure VOC/gas levels and *estimate* CO2

- Problem: Affected by humidity, temperature, and every other gas in the room
- Accuracy: ± 500 -1000 ppm easily

NDIR sensors (MH-Z19C): Fire infrared light through an air sample

- CO2 absorbs infrared at a specific wavelength
- Measure the absorption $\rightarrow$ that IS the CO2 concentration
- Accuracy: ± 50 ppm

Why it matters for my project: I'm making health decisions based on CO2 readings.

An estimated value ± 500 ppm is clinically meaningless. Real NDIR measurement is

Communication Protocol 3: Analog

The Simplest Protocol — With Its Own Challenges

The MS1100 VOC sensor outputs a **voltage** proportional to gas concentration.

```
ESP32-C6 A0 (D0) ← MS1100 OUT
VBUS (5V)        → MS1100 VCC
```

Reading:

```
int rawVOC = analogRead(A0); // 0-4095 (12-bit ADC)
float voltage = rawVOC * (3.3 / 4095.0);
```

What AI taught me: The ESP32 ADC is non-linear at the extremes (near 0V and 3.3V).
Use readings in the middle range for reliability.

Sensor Calibration — Noise

Every Low-Cost Sensor Has Two Problems

1. **Noise** — readings fluctuate randomly second-to-second
2. **Drift** — baseline shifts slowly over days/weeks

Technique 1: Moving Average Filter

```
const int WINDOW = 10;
float readings[WINDOW];
int idx = 0;

float movingAverage(float newVal) {
    readings[idx % WINDOW] = newVal;
    idx++;
    float sum = 0;
    for (int i = 0; i < WINDOW; i++) sum += readings[i];
    return sum / WINDOW; // Average last 10 → smoother output
```

Sensor Calibration — Drift and Warm-up

Techniques 2 and 3

Technique 2: MH-Z19C ABC (Automatic Baseline Calibration)

- The sensor tracks its own minimum reading over 24 hours
- Uses that minimum to correct long-term drift — built into the chip
- No code required; happens automatically in the background

Technique 3: Warm-up Time

- CO2 and VOC sensors need **1–3 minutes** after power-on before readings stabilize
- Firmware skips publishing to Firebase until sensor has been running ≥ 90 seconds

```
// Skip first 90 seconds of readings  
if (millis() < 90000) return;
```

Running 5 Sensors Simultaneously

The Real Challenge: Timing and Memory

Each sensor has its own requirements:

Sensor	Read Frequency	Notes
AHT10	Every 2s	Needs 75ms after trigger
MH-Z19C	Every 10s	9-byte UART command/response
MS1100	Every 1s	Instant analog read
LCD1602	Every 3s	Rotates screens
DC01	Every 30s	Serial stream parsing

AI helped me: Structure the firmware with `millis()`-based timing instead of `delay()`

AI Interaction That Changed My Understanding

My question to Claude:

"Why does my CO2 sensor give 400 ppm constantly even in a closed room?"

AI's diagnosis:

- 400 ppm is atmospheric CO2 baseline (outdoor air)
- In a closed room without ventilation, CO2 should rise over time as I breathe
- The sensor may be in calibration mode, or UART wiring TX/RX may be swapped

What I did: Swapped TX and RX wires → immediately got readings rising to 1700+ ppm in my closed office.

This one conversation saved hours of debugging. It also taught me that **CO2 rising in a closed room is the expected behavior — the purifier sensor showing 400 ppm always is the problem I'm solving**

What I Learned — Summary

Before	After
"I2C? That's some bus thing"	Can wire multiple devices, manage addresses, debug conflicts
"UART just means serial"	Can write command/response protocols, parse byte arrays
"Sensors just give numbers"	Understand noise, drift, warm-up, calibration, accuracy classes
"The ESP32 is the ESP32"	Know that every board variant has different pin mappings — always verify
"AI knows everything"	AI knows common things. For edge cases, always verify with datasheets

Key Takeaway

Sensors are not black boxes. They are physical measurement instruments with specific electrical requirements, communication protocols, timing constraints, and accuracy limitations — and every one of those things must be understood and managed to get reliable data.

AI accelerated my learning dramatically, but the real understanding came from:

1. Reading official datasheets
2. Writing the driver code myself with AI guidance
3. Debugging when things didn't work — and understanding *why*

Repository: <https://github.com/shresthamausam07/smart-home-health-monitor>

Thank You

Mausam Shrestha

CSC 494 — IoT | Spring 2026

shrestham2@mymail.nku.edu

Topic 1 complete — Multi-Sensor Integration & Calibration

Next: Topic 2 — Mobile App Development with React/PWA